Attribute & Deontic Matching

How **tag_attribute.py** and **tag_deontic.py** convert Excel-coded Institutional Grammar fields into precise character offsets inside the official plain-text version of the norma.

1. The problem

Coders annotate each atomistic statement in an Excel spreadsheet (**PLAN_INSUMO.xlsx**). For every row they record, among other fields, an **Attribute** (the actor / holder) and a **Deontic** (the modal). The goal of this stage is to turn each of those Excel fields into a **brat**-style annotation — a byte offset range inside the plain text **document.txt** — so the annotations can be visualized and reviewed against the actual legal source.

That mapping is non-trivial because:

- The Excel *Raw Statement* is a paraphrase of text that lives in the document, often with different whitespace, line-wrapping, or minor capitalization.
- The *Attribute* is frequently written with **square brackets** around implied words (e.g. `"[Titulares de] fuentes estacionarias"`) — those brackets are metadata, not literal text in the document.
- The *Deontic*'s surface phrase (e.g. `"deberán"`, or a paraphrase like `"prohíbe la operación"`) is captured from the **bold run** inside the Excel Raw Statement cell and must be located inside the corresponding stretch of document.txt.

2. Inputs and outputs

Inputs (produced by earlier stages):

```
outputs/document.txt      # plain text of the norma
outputs/statements.json   # one entry per Excel row, e.g.:
  {
    "stmt_no": 42,
    "raw_statement": "Los titulares de fuentes estacionarias deberan ...",
    "attribute":     "[Titulares de] fuentes estacionarias",
    "deontic":       "deberan",
    "deontic_surface":"deberan"
  }
```

Outputs:

```
outputs/annotations/document.ann              # brat annotations, surface text
outputs/annotations/deontic_literal.ann       # same, but Deontic text = Excel value
outputs/unmatched.csv, unmatched_deontic.csv, unmatched_statements.csv
```

3. Pipeline at a glance

For each statement (in **stmt_no** order), the tagger does three things, in this order:

```
for stmt in statements:
    (a) span   = find_statement_span(doc, stmt.raw_statement, cursor)
```

```
(b) attr   = match_attribute(doc[span], stmt.attribute)
             or match_attribute_cross_reference(doc, stmt.attribute, span.start)
(c) deont  = tag_deontic_for_span(doc, stmt, span.start, span.end)
cursor    = span.end   # move forward; do not re-scan already-tagged prefix
```

The **cursor** is important: statements are processed in the same order they appear in the document, and each next search starts at the previous statement's end. That prevents an earlier repeated phrase from stealing the match away from a later, correct occurrence.

4. Matching primitives (matching.py)

4.1 normalize_with_map

Collapses every run of whitespace (spaces, newlines, tabs) into a single space and strips ends. Alongside the normalized string it returns an **index map** — a list where entry *i* is the offset in the *original* string that normalized character *i* came from. That is what allows any match found in the normalized text to be translated back into precise offsets in the raw **document.txt**.

```
raw:        "Los  titulares\n  de fuentes"
normalized: "Los titulares de fuentes"
index_map:  [0, 1, 2, 3, 5, 6, ..., 13, 16, 17, 19, ...]  # back to raw offsets
```

4.2 find_exact

Plain `str.find`. Fast path for the common case where the Excel text is a byte-for-byte substring of the document.

4.3 find_whitespace_insensitive

Normalizes both haystack and needle with **normalize_with_map**, searches in the normalized haystack, then uses the index map to project the match back to raw offsets. Optional **ignore_case** handles the case where Excel's canonical capitalization ("Buses") differs from the document ("los buses ...").

4.4 reject_multiline

A brat annotation is a single line. If a whitespace-insensitive match spans a raw newline, the annotation cannot be written — so the match is rejected and the caller falls further down the cascade instead of producing a broken output.

4.5 find_last_occurrence_before

Used only for the cross-reference fallback. Given a target string and an upper-bound offset, it finds the **latest** occurrence of that string that ends at or before the bound — walking backward through the document and skipping any multi-line hit. This lets an Attribute that isn't in the current statement's own text be resolved to the nearest previous mention.

5. Locating the statement span

Before either the Attribute or the Deontic can be tagged, the **raw_statement** text itself must be located inside document.txt. **find_statement_span** tries, in order:

#	Strategy	What it does
1	exact from cursor	str.find(raw_statement, cursor) — most common case.
2	exact from doc start	Retry from offset 0 in case the ordering assumption is wrong.
3	whitespace-insensitive	Normalize both sides; find first match at or after cursor, then fall back to first match an

Statements that still cannot be located are dropped into **unmatched_statements.csv** — no Attribute or Deontic will be tagged for them.

6. Attribute matching

6.1 Local cascade (inside the statement span)

match_attribute tries the following strategies against the sliced `doc_text[start:end]`, stopping at the first one that finds a match whose text does not embed a newline:

#	Strategy	What it does
1	exact	Literal substring of the statement span.
2	exact, brackets stripped	Remove any […] segments from the Excel value first (e.g
3	whitespace-insensitive	Same as (1) but tolerant to newline/whitespace differences.
4	whitespace-insensitive, stripped	Same as (2) but tolerant to whitespace.
5	whitespace-insensitive, case-insensitive	Handles "Buses" (Excel) vs "buses" (document).
6	whitespace-insensitive, stripped, ci	Both accommodations at once — last resort.

6.2 Cross-reference fallback (whole-document lookup)

When the Attribute contains **square brackets** and none of the six local strategies work, coders were signalling that the bracketed material is implied. Concretely, most rows of the form `"[Titulares de] X"` or `"Titulares de [X]"` mean: the actor **X** (or the role phrase) was already named explicitly in an earlier sentence of the same article, and this atomistic statement doesn't restate it.

match_attribute_cross_reference builds a small list of candidate strings and searches **backward** from the statement start through the entire document, taking the nearest previous occurrence:

```
candidates = []
outside = brackets_stripped(attribute)               # e.g. "fuentes estacionarias"
if outside and not generic_role_only(outside):
    candidates.append(outside)                       # #1 primary

for segment in bracket_contents(attribute):          # e.g. "Titulares de"
    if segment and segment not in {"and", "or"} and not generic_role_only(segment):
        candidates.append(segment)
    object_part = strip_role_prefix(segment)         # "Titulares de calderas" -> "calderas"
    if object_part and object_part != segment and not generic_role_only(object_part):
        candidates.append(object_part)

for candidate in candidates:
    hit = find_last_occurrence_before(doc, doc_norm, doc_norm_map,
                                      candidate, before_index=stmt_start)
    if hit: return hit
```

A guard, _**is_generic_role_only**, discards candidates like "Titulares de" or "Usuarios" with no object — those role words recur throughout legal text and a backward search would confidently land on an unrelated sentence rather than falling through to "no match". **find_last_occurrence_before** also rejects any candidate hit that would span a newline (it retries further back).

When a cross-reference match is used, the annotation is stamped with `match_source = "cross_reference"`; local matches are stamped `"local"`. That flag is what **attribute_review.py** uses to prioritize human review.

7. Deontic matching

The Deontic side is simpler than the Attribute side. The Excel *Raw Statement* cell marks the surface phrase that carries the modality with **bold** formatting; the earlier extraction stage stores that bold run under **deontic_surface**. Sometimes it is the modal itself ("deberán", "podrán"), sometimes a future-tense verb that stands in for an implied `[deberá]` ("generará"), and sometimes a paraphrase ("prohíbe la operación", "será fiscalizada").

tag_deontic_for_span is called with the statement's already-resolved `(start, end)` — no need to re-locate the statement. It then runs **match_deontic**, whose cascade mirrors **match_attribute** but without the cross-reference fallback (the surface phrase, being bold in the Excel Raw Statement, is by construction present in this statement's own text):

#	Strategy	What it does
1	exact	Literal substring of the statement span.
2	exact, brackets stripped	Belt-and-suspenders: a bold run shouldn't contain "[…]" but a stray bracket must not c
3	whitespace-insensitive	Tolerant to newline/whitespace differences.
4	whitespace-insensitive, stripped	Both accommodations at once.

Two rules on when a Deontic is tagged at all:

- If the Excel **Deontic** cell is blank, the row is skipped outright — an empty coding is not an unmatched deontic.
- If **deontic_surface** is empty, likewise skipped.
- Otherwise the cascade runs; failure lands in **unmatched_deontic.csv**.

Every deontic hit is tagged with the same brat type, plain **Deontic**. The Excel *Deontic* value itself (e.g. "deberán", "[podrán]") is kept as the annotation's **deontic** field, not folded into the type name — that keeps the brat schema flat and lets the modal token stay verbatim, including its `[…]` brackets that mark implicit modality.

8. Output files

Two **.ann** files are written per document (both share the same spans and only differ in the text stored on each Deontic line):

```
# document.ann  — surface text, i.e. what the document actually says
T1  Attribute 152 189   los titulares de fuentes estacionarias
T2  Deontic   190 197   deberan

# deontic_literal.ann  — Attribute lines identical; Deontic text is the
#                        canonical Excel value (may include square brackets)
T1  Attribute 152 189   los titulares de fuentes estacionarias
T2  Deontic   190 197   [debera]
```

Both files list all Attribute annotations first (in statement order) and all Deontic annotations after, each with a unique `T<n>` id. That is the exact layout brat expects: one span per line, tab-separated,

`<type> <start> <end> <text>`.

9. Failure buckets

Anything that cannot be resolved is written to a CSV so a human can review it — the pipeline never guesses:

#	Strategy	What it does
1	unmatched_statements.csv	Raw statement not found in document.txt at all.
2	unmatched.csv	Statement found but its Attribute couldn't be located locally, and (if bracketed) the cro
3	unmatched_deontic.csv	Statement found but its bold deontic_surface couldn't be located in the span.